Neuro-formal verification (arXiv:2608.21516) · replication demo

One benchmark entry, every call

HumanEval/42 `incr_list`, its correct version and a buggy mutant, through the staged Python → Dafny pipeline with **Claude Opus** (`claude -p`, isolated) and the **Dafny 4.11** verifier. Recorded live: 12 LLM calls, 12 verifier runs.

correct (reference)

```python
def incr_list(l: list):
    return [x + 1 for x in l]
```

buggy (mutant)

```python
def incr_list(l: list):
    for i in range(len(l)):
        # The buggy line below adds 2 instead of 1, introducing an off-by-one error to the expected result
        l[i] += 2

    return l
```

postcondition S_L (both): `assert all((return_value[i] == l[i] + 1 for i in range(len(l))))`

model **claude-opus** · effort medium

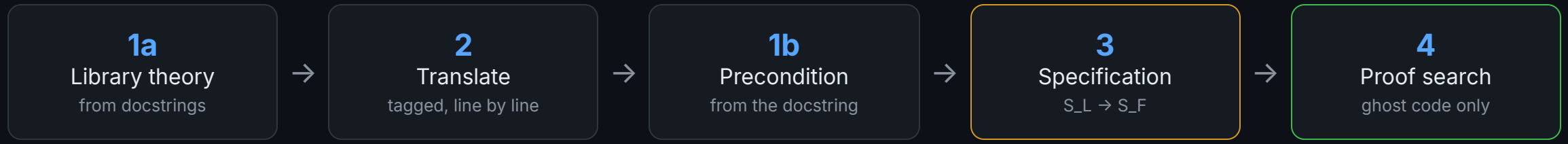
isolation **--safe-mode**, no tools, empty cwd

budget **2 attempts × 4 rounds**

verifier **Dafny 4.11** (Z3)

recorded 2026-10-15

The staged, goal-blind pipeline



← stages before 3 never see the property (goal-blind) →

- Every stage is one `claude -p` call; each answer is checked by `dafny resolve` (syntax/types) and mechanical audits before the next stage runs.
- Refutation reuses the same stages, negates the goal, asks for a **witness** input, and must **prove** the violation, then pass a **reachability** check.
- Frozen during proof search: library theory, spec, every `// src:` line and its code, the contract. Only ghost code may be added, which is checked by ghost-erasure.

Correct program

refutation arm: look for a bug first

Pipeline order (paper, Table 3): first try to **refute** (find and prove a counterexample), then try to **verify** what is left.

```
def incr_list(l: list):
    return [x + 1 for x in l]
```

S_L: `assert all((return_value[i] == l[i] + 1 for i in range(len(l))))`

LLM CALL **dafny.theory** Formalize the Python library surface the program calls (E_F), from the docs.

WHAT THIS STAGE SEES

Python source | library docstrings | never sees: postcondition S_L

Python program (source.py)

```
1 | def incr_list(l: list):
2 |     """Return list with elements incremented by 1.
3 |     >>> incr_list([1, 2, 3])
4 |     [2, 3, 4]
5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
7 |     """
8 |     return [x + 1 for x in l]
```

OPUS ANSWERS 4.2s · $0.004

> The program uses only a list comprehension and integer addition, so it needs no library theory.

```
// No library calls beyond the core language (list comprehension, integer addition).
```

✓ **1a.theory**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.translate** Translate the program line by line into Dafny; every line tagged src:/synth:.

WHAT THIS STAGE SEES

Python source (numbered) | frozen theory | never sees: postcondition S_L

Python program (source.py)

```
1 | def incr_list(l: list):
2 |     """Return list with elements incremented by 1.
3 |     >>> incr_list([1, 2, 3])
4 |     [2, 3, 4]
5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
7 |     """
8 |     return [x + 1 for x in l]
```

OPUS ANSWERS 7.5s · $0.011

> The parameter is modeled as `seq<int>` because the docstring examples only use integers. The list comprehension becomes an index-based `while` loop that appends `x + 1` to an accumulator.

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
{
  // synth: translation helper, accumulator for the list comprehension
  var result: seq<int> := [];
  var i: int := 0;
  // src: source.py:8 | return [x + 1 for x in l]
  while i < |l|
  {
    // src: source.py:8 | return [x + 1 for x in l]
    var x := l[i];
    result := result + [x + 1];
    // synth: translation helper, loop index increment
    i := i + 1;
  }
  // src: source.py:8 | return [x + 1 for x in l]
  return_value := result;
  return;
}
```

✓ `dafny resolve` well-formed · 0.45s

✓ **2.translate**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.precondition** State the input assumptions the docstring makes, as `requires`.

WHAT THIS STAGE SEES

docstring signatures theory never sees: S_L translation body

signature + docstring

```
def incr_list(l: list):
    """Return list with elements incremented by 1.
>>> incr_list([1, 2, 3])
[2, 3, 4]
>>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
[6, 4, 6, 3, 4, 4, 10, 1, 124]"""
```

OPUS ANSWERS 5.8s · $0.004

> The docstring states no input assumptions. The empty list and any integer values are valid inputs, and Dafny `int` is unbounded, so overflow cannot occur.

```
// none
```

✓ `dafny resolve` well-formed · 0.43s

✓ **1b.precondition**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.spec** Translate the Python postcondition S_L into Dafny `ensures` (S_F), faithfully.

WHAT THIS STAGE SEES

S_L · docstring · signatures · theory · never sees: translation body

Python postcondition S_L

```
assert all((return_value[i] == l[i] + 1 for i in range(len(l))))

# Dafny signature:
# method incr_list(l: seq<int>) returns (return_value: seq<int>)
```

OPUS ANSWERS

8.4s · $0.012

In Python, `return_value[i]` with `0 <= i < len(l)` raises `IndexError` if `return_value` is shorter than `l`, and the assertion then fails. So the length lower bound is part of S_L. S_L does not require the lengths to be equal: a longer `return_value` still passes.

```
ensures |return_value| >= |l|
ensures forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1
```

✓ `dafny resolve` well-formed · 0.48s

✓ **3.spec**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.witness** Propose an input that violates S_L (or answer NONE if there is none).

WHAT THIS STAGE SEES

Python source | S_L | precondition | never sees: Dafny translation body

Python program (source.py)

```
  1 | def incr_list(l: list):
  2 |     """Return list with elements incremented by 1.
  3 |     >>> incr_list([1, 2, 3])
  4 |     [2, 3, 4]
  5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
  6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
  7 |     """
  8 |     return [x + 1 for x in l]

# postcondition S_L
assert all((return_value[i] == l[i] + 1 for i in range(len(l))))
```

OPUS ANSWERS 5.3s · $0.004

> The program is correct for every input, so no counterexample exists.

```
// NONE
```

✓ **witness**: audits passed (provenance tags, frozen parts, no assumptions)

✓ **witness**: model answered NONE → refutation abstains (no proof budget spent)

ABSTAIN

the model found no counterexample → the pipeline moves on to verification

refute arm · 32.4s wall time

Correct program

verification arm: prove it correct

Refutation found nothing, so the pipeline now tries to **prove** the program correct.

S_L: `assert all((return_value[i] == l[i] + 1 for i in range(len(l))))`

Stages 1a, 2, 1b and 3 are reused byte-identically from the refutation arm (same attempt): only the goal changes.

1a Library theory | 2 Translate | 1b Precondition | 3 Specification | W ~~Witness~~ | 4 Proof search | R ~~Reachability~~ | ✓ Verdict

DAFNY **Proof attempt 0** The harness assembles THEORY + SPEC + PROGRAM with the frozen contract and runs the verifier before asking for any proof.

INPUT: ARTIFACT.DFY (PROGRAM REGION)

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  // synth: frozen postcondition (stage 3)
  ensures |return_value| >= |l|
  // synth: frozen postcondition (stage 3)
  ensures forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1
{
  // synth: translation helper, accumulator for the list comprehension
  var result: seq<int> := [];
  var i: int := 0;
  // src: source.py:8 | return [x + 1 for x in l]
  while i < |l|
  {
    // src: source.py:8 | return [x + 1 for x in l]
    var x := l[i];
    result := result + [x + 1];
    // synth: translation helper, loop index increment
    i := i + 1;
  }
  // src: source.py:8 | return [x + 1 for x in l]
  return_value := result;
  return;
}
```

`dafny verify --allow-warnings --verification-time-limit 30 --cores 2 artifact.dfy` 1.01s

```
artifact.dfy(29,2): Error: a postcondition could not be proved on this return path
   |
29 |   return;
   |   ^^^^^^^
artifact.dfy(11,25): Related location: this is the postcondition that could not be
proved
   |
11 |   ensures |return_value| >= |l|
   |                           ^^
artifact.dfy(29,2): Error: a postcondition could not be proved on this return path
   |
29 |   return;
   |   ^^^^^^^
artifact.dfy(13,10): Related location: this is the postcondition that could not be
proved
   |
13 |   ensures forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1
   |           ^^^^^^
Dafny program verifier finished with 1 verified, 2 errors
```

× not proved yet: errors go back to the model (stage 4)

1a Library theory | 2 Translate | 1b Precondition | 3 Specification | W ~~Witness~~ | 4 Proof search | R ~~Reachability~~ | ✓ Verdict

LLM CALL `dafny.proof` Proof search: add ghost proof code only (invariants, asserts, lemmas). Everything else is frozen.

WHAT THIS STAGE SEES

assembled artifact | verifier errors | never sees: - (may only add ghost code)

verifier output fed back

```
artifact.dfy(29,2): Error: a postcondition could not be proved on
this return path
   |
29 |   return;
   |   ^^^^^^^

artifact.dfy(11,25): Related location: this is the postcondition that
could not be proved
   |
11 |   ensures |return_value| >= |l|
   |                           ^^

artifact.dfy(29,2): Error: a postcondition could not be proved on
this return path
   |
29 |   return;
   |   ^^^^^^^

artifact.dfy(13,10): Related location: this is the postcondition that
could not be proved
   |
13 |   ensures forall j :: 0 <= j < |l| ==> return_value[j] == l[j] +
```

OPUS ANSWERS 9.5s · $0.030

> I added loop invariants for the index bounds, the accumulator length and the element-wise relation, plus a `decreases` clause. Together with the loop exit condition, these give both postconditions.

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  ⋮
  while i < |l|
    // synth: invariant
    invariant 0 <= i <= |l|
    // synth: invariant
    invariant |result| == i
    // synth: invariant
    invariant forall j :: 0 <= j < i ==> result[j] == l[j] + 1
    // synth: decreases
    decreases |l| - i
  {
```

green = lines added by proof search (8 lines, all ghost code); ⋮ = unchanged frozen code

✓ `dafny verify` verified · 0.98s | ✓ **4.proof**: proof checked

1a Library theory | 2 Translate | 1b Precondition | 3 Specification | W ~~Witness~~ | 4 Proof search | R ~~Reachability~~ | ✓ Verdict

VERIFIED

machine-checked proof that the program satisfies S_F under E_F

ground-truth label: correct → correct verdict

verify arm · 11.5s wall time

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  // synth: frozen postcondition (stage 3)
  ensures |return_value| >= |l|
  // synth: frozen postcondition (stage 3)
  ensures forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1
{
  // synth: translation helper, accumulator for the list comprehension
  var result: seq<int> := [];
  var i: int := 0;
  // src: source.py:8 | return [x + 1 for x in l]
  while i < |l|
    // synth: invariant
    invariant 0 <= i <= |l|
    // synth: invariant
    invariant |result| == i
    // synth: invariant
    invariant forall j :: 0 <= j < i ==> result[j] == l[j] + 1
    // synth: decreases
    decreases |l| - i
  {
    // src: source.py:8 | return [x + 1 for x in l]
    var x := l[i];
    result := result + [x + 1];
    // synth: translation helper, loop index increment
    i := i + 1;
  }
… 4 more lines
```

Buggy program

refutation arm: look for a bug first

Pipeline order (paper, Table 3): first try to **refute** (find and prove a counterexample), then try to **verify** what is left.

```python
def incr_list(l: list):
    for i in range(len(l)):
        # The buggy line below adds 2 instead of 1, introducing an off-by-one error to the expected result
        l[i] += 2

    return l
```

S_L: `assert all((return_value[i] == l[i] + 1 for i in range(len(l))))`

LLM CALL **dafny.theory** Formalize the Python library surface the program calls (E_F), from the docs.

WHAT THIS STAGE SEES

Python source | library docstrings | never sees: postcondition S_L

Python program (source.py)

```
  1 | def incr_list(l: list):
  2 |     """Return list with elements incremented by 1.
  3 |     >>> incr_list([1, 2, 3])
  4 |     [2, 3, 4]
  5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
  6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
  7 |     """
  8 |
  9 |     for i in range(len(l)):
 10 |         # The buggy line below adds 2 instead of 1, introducing
an off-by-one error to the expected result
 11 |         l[i] += 2
 12 |
 13 |     return l
```

OPUS ANSWERS 6.0s · $0.006

The program calls only `len` and `range`, indexes and assigns list elements, and adds integers. These map directly to Dafny's `|s|`, quantification over indices, sequence update and `int` arithmetic. Under the stage rules, the theory is therefore empty.

```
// The program uses only len, range, list indexing and integer addition; no
library theory is required.
```

✓ **1a.theory**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.translate** Translate the program line by line into Dafny; every line tagged src:/synth:.

WHAT THIS STAGE SEES

Python source (numbered) | frozen theory | never sees: postcondition S_L

Python program (source.py)

```
  1 | def incr_list(l: list):
  2 |     """Return list with elements incremented by 1.
  3 |     >>> incr_list([1, 2, 3])
  4 |     [2, 3, 4]
  5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
  6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
  7 |     """
  8 |
  9 |     for i in range(len(l)):
 10 |         # The buggy line below adds 2 instead of 1, introducing
an off-by-one error to the expected result
 11 |         l[i] += 2
 12 |
 13 |     return l
```

OPUS ANSWERS 8.7s · $0.026

This keeps the source's `+= 2` exactly as written; it does not fix the docstring's expected +1. Python mutates the caller's list in place, but this is modeled as a value-semantics `seq` returned as `return_value`.

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
{
  // synth: declaration, mutable local copy of the list parameter (Python
mutates l in place)
  var l' := l;
  // src: source.py:9 | for i in range(len(l)):
  var n := |l'|;
  var i := 0;
  while i < n
  {
    // src: source.py:11 | l[i] += 2
    l' := l'[i := l'[i] + 2];
    // synth: loop increment, for the range iteration
    i := i + 1;
  }
  // src: source.py:13 | return l
  return_value := l';
}
```

✓ `dafny resolve` well-formed · 0.44s

✓ **2.translate**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.precondition** State the input assumptions the docstring makes, as `requires`.

WHAT THIS STAGE SEES

docstring signatures theory never sees: S_L translation body

signature + docstring

```
def incr_list(l: list):
    """Return list with elements incremented by 1.
>>> incr_list([1, 2, 3])
[2, 3, 4]
>>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
[6, 4, 6, 3, 4, 4, 10, 1, 124]"""
```

OPUS ANSWERS 4.3s · $0.010

> The docstring states no input assumptions. Any list of integers, including the empty list, is valid input.

```
// none
```

✓ `dafny resolve` well-formed · 0.45s

✓ **1b.precondition**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.spec** Translate the Python postcondition S_L into Dafny `ensures` (S_F), faithfully.

WHAT THIS STAGE SEES

S_L · docstring · signatures · theory · never sees: translation body

Python postcondition S_L

```
assert all((return_value[i] == l[i] + 1 for i in range(len(l))))

# Dafny signature:
# method incr_list(l: seq<int>) returns (return_value: seq<int>)
```

OPUS ANSWERS 6.9s · $0.021

> The first clause is needed because the Python assertion raises `IndexError` (and so fails) if `return_value` is shorter than `l`. The assertion says nothing about extra trailing elements, so a longer result is not ruled out.

```
ensures |return_value| >= |l|
ensures forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1
```

✓ `dafny resolve` well-formed · 0.47s

✓ **3.spec**: audits passed (provenance tags, frozen parts, no assumptions)

LLM CALL **dafny.witness** Propose an input that violates S_L (or answer NONE if there is none).

WHAT THIS STAGE SEES

Python source | S_L | precondition | never sees: Dafny translation body

Python program (source.py)

```
  1 | def incr_list(l: list):
  2 |     """Return list with elements incremented by 1.
  3 |     >>> incr_list([1, 2, 3])
  4 |     [2, 3, 4]
  5 |     >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
  6 |     [6, 4, 6, 3, 4, 4, 10, 1, 124]
  7 |     """
  8 |
  9 |     for i in range(len(l)):
 10 |         # The buggy line below adds 2 instead of 1, introducing
an off-by-one error to the expected result
 11 |         l[i] += 2
 12 |
 13 |     return l

# postcondition S_L
assert all((return_value[i] == l[i] + 1 for i in range(len(l))))
```

OPUS ANSWERS 6.8s · $0.020

> On this input the function returns `[3]`, but S_L requires `[2]`. In the Python version the list is mutated in place, so `return_value[i] == l[i]` and `l[i] + 1` can never equal it. In the Dafny model `l` is immutable, so the result is `l[i] + 2`, which is never `l[i] + 1`. So the postcondition fails for any non-empty list.

```
requires l == [1]
```

✓ `dafny resolve` well-formed · 0.48s

✓ **witness**: audits passed (provenance tags, frozen parts, no assumptions)

DAFNY

Proof attempt 0

The harness assembles THEORY + SPEC + PROGRAM with the frozen contract and runs the verifier before asking for any proof.

INPUT: ARTIFACT.DFY (PROGRAM REGION)

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  // synth: witness (refutation)
  requires l == [1]
  // synth: frozen negated postcondition (refutation)
  ensures !((|return_value| >= |l|) && (forall i :: 0 <= i < |l| ==>
return_value[i] == l[i] + 1))
{
  // synth: declaration, mutable local copy of the list parameter (Python
mutates l in place)
  var l' := l;
  // src: source.py:9 | for i in range(len(l)):
  var n := |l'|;
  var i := 0;
  while i < n
  {
    // src: source.py:11 | l[i] += 2
    l' := l'[i := l'[i] + 2];
    // synth: loop increment, for the range iteration
    i := i + 1;
  }
  // src: source.py:13 | return l
  return_value := l';
}
```

dafny verify --allow-warnings --verification-time-limit 30 --cores 2 artifact.dfy 1.00s

```
artifact.dfy(14,0): Error: a postcondition could not be proved on this return path
   |
14 | {
   | ^
artifact.dfy(13,10): Related location: this is the postcondition that could not be
proved
   |
13 |   ensures !((|return_value| >= |l|) && (forall i :: 0 <= i < |l| ==>
return_value[i] == l[i] + 1))
   |           ^
artifact.dfy(23,13): Error: index out of range
   |
23 |     l' := l'[i := l'[i] + 2];
   |             ^
Dafny program verifier finished with 1 verified, 2 errors
```

× not proved yet: errors go back to the model (stage 4)

LLM CALL **dafny.refute.proof** Prove the NEGATED postcondition on the witness input. Code and goal are frozen.

WHAT THIS STAGE SEES

assembled artifact | verifier errors | never sees: - (may only add ghost code)

verifier output fed back

```
artifact.dfy(14,0): Error: a postcondition could not be proved on
this return path
   |
14 | {
   | ^

artifact.dfy(13,10): Related location: this is the postcondition that
could not be proved
   |
13 |   ensures !((|return_value| >= |l|) && (forall i :: 0 <= i < |l|
==> return_value[i] == l[i] + 1))
   |           ^

artifact.dfy(23,13): Error: index out of range
   |
23 |     l' := l'[i := l'[i] + 2];
   |             ^


Dafny program verifier finished with 1 verified, 2 errors
```

OPUS ANSWERS 8.5s · $0.034

> The loop needed invariants for the length and element values; the final asserts then show `return_value[0] == 3 != 2 == l[0] + 1`.

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  ⋮
  while i < n
    // synth: invariant
    invariant 0 <= i <= n
    // synth: invariant
    invariant n == |l'| == |l|
    // synth: invariant
    invariant forall j :: 0 <= j < i ==> l'[j] == l[j] + 2
    // synth: invariant
    invariant forall j :: i <= j < n ==> l'[j] == l[j]
    // synth: decreases
    decreases n - i
  {
  ⋮
  return_value := l';
  // synth: proof
  assert return_value[0] == 3;
  // synth: proof
  ⋮ (14 lines added in total)
```

green = lines added by proof search (14 lines, all ghost code); ⋮ = unchanged frozen code

✓ `dafny verify` verified · 0.97s | ✓ **4.proof**: proof checked

DAFNY

Reachability check

The proved artifact again, with the goal replaced by `ensures false`. It must NOT verify, otherwise the witness would be vacuous (contradicting the precondition).

INPUT: ARTIFACT.DFY (PROGRAM REGION)

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  // synth: witness (refutation)
  requires l == [1]
  // synth: frozen negated postcondition (refutation)
  ensures false
{
  // synth: declaration, mutable local copy of the list parameter (Python
mutates l in place)
  var l' := l;
  // src: source.py:9 | for i in range(len(l)):
  var n := |l'|;
  var i := 0;
  while i < n
    // synth: invariant
    invariant 0 <= i <= n
    // synth: invariant
    invariant n == |l'| == |l|
    // synth: invariant
    invariant forall j :: 0 <= j < i ==> l'[j] == l[j] + 2
    // synth: invariant
    invariant forall j :: i <= j < n ==> l'[j] == l[j]
    // synth: decreases
    decreases n - i
  {
    // src: source.py:11 | l[i] += 2
… 9 more lines
```

dafny verify --allow-warnings --verification-time-limit 30 --cores 2 artifact.dfy 0.99s

```
artifact.dfy(14,0): Error: a postcondition could not be proved on this return path
   |
14 | {
   | ^
artifact.dfy(13,10): Related location: this is the postcondition that could not be
proved
   |
13 |   ensures false
   |           ^^^^^
Dafny program verifier finished with 0 verified, 1 error
```

✓ fails as required: the witness is a real, reachable input

REFUTED

machine-checked proof that the program violates S_F on the witness

witness: `l == [1]`

ground-truth label: incorrect → correct verdict

refute arm · 46.0s wall time

```
// src: source.py:1 | def incr_list(l: list):
method incr_list(l: seq<int>) returns (return_value: seq<int>)
  // synth: witness (refutation)
  requires l == [1]
  // synth: frozen negated postcondition (refutation)
  ensures !((|return_value| >= |l|) && (forall i :: 0 <= i < |l| ==> return_value[i] == l[i] + 1))
{
  // synth: declaration, mutable local copy of the list parameter (Python mutates l in place)
  var l' := l;
  // src: source.py:9 | for i in range(len(l)):
  var n := |l'|;
  var i := 0;
  while i < n
    // synth: invariant
    invariant 0 <= i <= n
    // synth: invariant
    invariant n == |l'| == |l|
    // synth: invariant
    invariant forall j :: 0 <= j < i ==> l'[j] == l[j] + 2
    // synth: invariant
    invariant forall j :: i <= j < n ==> l'[j] == l[j]
    // synth: decreases
    decreases n - i
  {
    // src: source.py:11 | l[i] += 2
    l' := l'[i := l'[i] + 2];
    // synth: loop increment, for the range iteration
… 9 more lines
```

Result: both verdicts correct, both machine-checked

entry	arm	verdict	witness	time
HumanEval_42/reference	refute	abstain		32.4s
HumanEval_42/reference	verify	verified		11.5s
HumanEval_42/mutant	refute	refuted	`l == [1]`	46.0s

12	82s	12	8.2s	$0.18
LLM calls	model time	Dafny runs	verifier time	list-price cost

Full trace with every prompt, response and verifier output: `demo/HumanEval_42/trace.md` · raw: `demo/HumanEval_42/run/trace.jsonl` · reproduce: `nfv run --arm pipeline --tasks 42 --trace`